

Automata Theory & Compiler Design (AT & CD)

Syllabus

UNIT-I: Strings, Alphabet, Language, Operations, finite automaton model, acceptance of strings, and languages, deterministic finite automaton and non deterministic finite automaton, Equivalence between NFA to DFA conversion.

UNIT-II: Regular Languages, Regular sets, regular expressions, Constructing finite Automata for a given regular expressions, Conversion of Finite Automata to Regular expressions. closure properties of regular sets (proofs not required).

Context Free Grammars: Context free grammar, derivation trees, Right most and leftmost derivation of strings. Ambiguity in context free grammars. Minimization of Context Free Grammars. Chomsky normal form, Greiback normal form,

UNIT-III: Push down automata: definition, model, acceptance of CFL, Introduction to DCFL and DPDA.

Turing Machine : Turing Machine, definition, model, design of TM, recursively enumerable languages. Chomsky hierarchy of languages

UNIT VI: Overview of compiler – Environment, pass, phase, phases of compiler, LEX tool,

Top Down Parsing: Top down parsing technique, Recursive decent parsing with back tracking, Ambiguous grammar, Elimination of left recursion, Left factoring, Predictive parsing, LL(1).

UNIT V: Bottom up parsing: shift reduce parser SLR, CLR, LALR, operator precedence parser, LR (O), LR(1), LR(K) grammar, YACC tool.

UNIT VI: Semantic Analysis: Syntax directed translation, S- Attributed, L Attributed definition, Type checker, Intermediate code generation: 3-address code form, DAG. Code optimization: Optimization, loop optimization, peep-hole optimization, Symbol table format

TEXTBOOKS:

1. Introduction to Automata Theory Languages and Computation. Hopcroft H.E. and Ullman J. D. Pearson Education
2. Introduction to Theory of Computation? Sipser 2nd edition Thomson
3. Compilers Principles, Techniques and Tools Aho, Ullman, Ravisethi, Pearson Education

UNIT-1

FORMAL LANGUAGE

LANGUAGES, DEFINITIONS

- Languages: different types of languages are:
- Natural languages
- Programming languages
- Mathematical languages etc..

Symbols :

Symbols are indivisible objects or entity that cannot be defined. That is, symbols are the atoms of the world of languages. A symbol is any single object such as $\clubsuit$, α , 0, 1, #, begin, or do. Usually, characters from a typical keyboard are only used as symbols.

Alphabets :

An alphabet is a finite, nonempty set of symbols. The alphabet of a language is normally denoted by Σ . When more than one alphabets are considered for discussion, then subscripts may be used (e.g. Σ_1, Σ_2 etc) or sometimes other symbol like $\mathcal{G}$ may also be introduced.

Example : $\Sigma = \{0, 1\}$

$$\Sigma = \{a, b, c\}$$

$$\Sigma = \{a, b, c, \&, z\}$$

$$\Sigma = \{\#, \nabla, \spadesuit, \beta\}$$

Strings or Words over Alphabet :

A string or word over an alphabet Σ is a finite sequence of concatenated symbols of Σ .

Example : 0110, 11, 001 are three strings over the binary alphabet $\{ 0, 1 \}$.

$aab, abcb, b, cc$ are four strings over the alphabet $\{ a, b, c \}$.

It is not the case that a string over some alphabet should contain all the symbols from the alphabet. For example, the string cc over the alphabet $\{ a, b, c \}$ does not contain the symbols a and b . Hence, it is true that a string over an alphabet is also a string over any superset of that alphabet.

Length of a string :

The number of symbols in a string w is called its length, denoted by $|w|$.

Example : $|011| = 4, |11| = 2, |b| = 1$

It is convenient to introduce a notation ϵ for the empty string, which contains no symbols at all. The length of the empty string ϵ is zero, i.e., $|\epsilon| = 0$.

Convention : We will use small case letters towards the beginning of the English alphabet to denote symbols of an alphabet and small case letters towards the end to denote strings over an alphabet. That is, $a, b, c \in \Sigma$ (symbols) and u, v, w, x, y, z are strings.

Some String Operations :

Let $x = a_1a_2a_3 \in a_n$ and $y = b_1b_2b_3 \in b_m$ be two strings. The concatenation of x and y denoted by xy , is the string $a_1a_2a_3 \cdots a_nb_1b_2b_3 \cdots b_m$. That is, the concatenation of x and y denoted by xy is the string that has a copy of x followed by a copy of y without any intervening space between them.

Example : Concatenation of the strings 0110 and 11 is 011011 and concatenation of the strings good and boy is goodboy.

Note that for any string w , $we = ew = w$. It is also obvious that if $|x| = n$ and $|y| = m$, then $|x + y| = n + m$.

u is a prefix of v if $v = ux$ for some string x .

u is a suffix of v if $v = xu$ for some string x .

u is a substring of v if $v = xuy$ for some strings x and y .

Example : Consider the string 011 over the binary alphabet. All the prefixes, suffixes and substrings of this string are listed below.

Prefixes: ϵ , 0, 01, 011.

Suffixes: ϵ , 1, 11, 011.

Substrings: ϵ , 0, 1, 01, 11, 011.

Note that x is a prefix (suffix or substring) to x , for any string x and ϵ is a prefix (suffix or substring) to any string.

A string x is a proper prefix (suffix) of string y if x is a prefix (suffix) of y and $x \neq y$.

In the above example, all prefixes except 011 are proper prefixes.

Powers of Strings : For any string x and integer $n \geq 0$, we use x^n to denote the string formed by sequentially concatenating n copies of x . We can also give an inductive definition of x^n as follows:

$$x^n = \epsilon, \text{ if } n = 0; \text{ otherwise } x^n = xx^{n-1}$$

Example : If $x = 011$, then $x^3 = 011011011$, $x^1 = 011$ and $x^0 = \epsilon$

Powers of Alphabets :

We write Σ^k (for some integer k) to denote the set of strings of length k with symbols from Σ . In other words, $\Sigma^k = \{ w \mid w \text{ is a string over } \Sigma \text{ and } |w| = k \}$. Hence, for any alphabet, Σ^0 denotes the set of all strings of length zero. That is, $\Sigma^0 = \{ \epsilon \}$. For the binary alphabet $\{ 0, 1 \}$ we have the following.

$$\Sigma^0 = \{ \epsilon \}.$$

$$\Sigma^1 = \{ 0, 1 \}.$$

$$\Sigma^2 = \{ 00, 01, 10, 11 \}.$$

$$\Sigma^3 = \{ 000, 001, 010, 011, 100, 101, 110, 111 \}$$

The set of all strings over an alphabet Σ is denoted by Σ^* . That is,

$$\begin{aligned}\Sigma^* &= \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots \cup \Sigma^k \cup \dots \\ &= \bigcup \Sigma^k\end{aligned}$$

The set Σ^* contains all the strings that can be generated by iteratively concatenating symbols from Σ any number of times.

Example : If $\Sigma = \{ a, b \}$, then $\Sigma^* = \{ e, a, b, aa, ab, ba, bb, aaa, aab, aba, abb, baa, \dots \}$.

Please note that if $\Sigma = \emptyset$, then Σ^* that is $\emptyset^* = \{ e \}$. It may look odd that one can proceed from the empty set to a non-empty set by iterated concatenation. But there is a reason for this and we accept this convention.

The set of all nonempty strings over an alphabet Σ is denoted by Σ^+ . That is,

$$\begin{aligned}\Sigma^+ &= \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots \cup \Sigma^k \cup \dots \\ &= \cup \Sigma^k\end{aligned}$$

Note that Σ^+ is infinite. It contains no infinite strings but strings of arbitrary lengths.

Reversal :

For any string $w = a_1a_2a_3 \dots a_n$ the reversal of the string is $w^R = a_na_{n-1} \dots a_3a_2a_1$.

An inductive definition of reversal can be given as follows:

Languages :

A language over an alphabet is a set of strings over that alphabet. Therefore, a language L is any subset of Σ^+ .

That is, any $L \subseteq \Sigma^+$ is a language.

Example :

1. $\varnothing$ is the empty language.
2. Σ^* is a language for any Σ .
3. $\{\epsilon\}$ is a language for any Σ . Note that, $\varnothing \neq \{\epsilon\}$. Because the language $\varnothing$ does not contain any string but $\{\epsilon\}$ contains one string of length zero.
4. The set of all strings over $\{0, 1\}$ containing equal number of 0's and 1's.
5. The set of all strings over $\{a, b, c\}$ that starts with a .

Convention : Capital letters A, B, C, L , etc. with or without subscripts are normally used to denote languages.

Set operations on languages : Since languages are set of strings we can apply set operations to languages. Here are some simple examples (though there is nothing new in it).

Union : A string $x \in L_1 \cup L_2$ iff $x \in L_1$ or $x \in L_2$

Example : $\{0, 11, 01, 011\} \cup \{1, 01, 110\} = \{0, 11, 01, 011, 111\} \quad \{0, 11, 01, 011, 1, 110\}$

Intersection : A string $x \in L_1 \cap L_2$ iff $x \in L_1$ and $x \in L_2$.

Example : $\{0, 11, 01, 011\} \cap \{1, 01, 110\} = \{01\}$

Complement : Usually, Σ^* is the universe that a complement is taken with respect to. Thus for a language L , the complement is $\bar{L} = \{x \in \Sigma^* \mid x \notin L\}$.

Example : Let $L = \{x \mid |x| \text{ is even}\}$. Then its complement is the language $\{x \in \Sigma^* \mid |x| \text{ is odd}\}$.

Similarly we can define other usual set operations on languages like relative complement, symmetric difference, etc.

Reversal of a language :

The reversal of a language L , denoted as L^R , is defined as: $L^R = \{w^R \mid w \in L\}$.

Example :

1. Let $L = \{0, 11, 01, 011\}$. Then $L^R = \{0, 11, 10, 110\}$.
2. Let $L = \{1^n 0^n \mid n \text{ is an integer}\}$. Then $L^R = \{1^n 0^n \mid n \text{ is an integer}\}$. $\{0^n 1^n\}$

Language concatenation : The concatenation of languages L_1 and L_2 is defined as

$$L_1 L_2 = \{xy \mid x \in L_1 \text{ and } y \in L_2\}.$$

Example : $\{a, ab\}\{b, ba\} = \{ab, aba, abb, abba\}$.

Note that ,

1. $L_1 L_2 \neq L_2 L_1$ in general.
2. $L\Phi = \Phi$
3. $L\{\varepsilon\} = L$

Iterated concatenation of languages : Since we can concatenate two languages, we also repeat this to concatenate any number of languages. Or we can concatenate a language with itself any number of times. The operation L^n denotes the concatenation of L with itself n times. This is defined formally as follows:

$$L_0 = \{\varepsilon\}$$

$$L^n = LL^{n-1}$$

Example : Let $L = \{a, ab\}$. Then according to the definition, we have

$$L_0 = \{\varepsilon\}$$

$$L_1 = L\{\varepsilon\} = L = \{a, ab\}$$

$$L_2 = L L_1 = \{a, ab\} \{a, ab\} = \{aa, aab, aba, abab\}$$

$$\begin{aligned} L_3 &= L L_2 = \{a, ab\} \{aa, aab, aba, abab\} \\ &= \{aaa, aaab, aaba, aabab, abaa, abaab, ababa, ababab\} \end{aligned}$$

and so on.

Kleene's Star operation : The Kleene star operation on a language L , denoted as L^* is defined as follows :

$$L^* = \left(\bigcup_{n \in \mathbb{N}} L^n \right)$$

$$= L^0 \cup L^1 \cup L^2 \cup \dots$$

$$= \{ x \mid x \text{ is the concatenation of zero or more strings from } L \}$$

Thus L^* is the set of all strings derivable by any number of concatenations of strings in L . It is also useful to define

$$L^+ = LL^*, \text{ i.e., all strings derivable by one or more concatenations of strings in } L. \text{ That is}$$

$$L^+ = \left(\bigcup_{n \in \mathbb{N} \text{ and } n > 0} L^n \right)$$

$$= L^1 \cup L^2 \cup L^3 \cup \dots$$

Example : Let $L = \{ a, ab \}$. Then we have,

$$L^* = L^0 \cup L^1 \cup L^2 \cup \dots$$

$$= \{e\} \cup \{a, ab\} \cup \{aa, aab, aba, abab\} \cup \dots$$

$$L^+ = L^1 \cup L^2 \cup L^3 \cup \dots$$

$$= \{a, ab\} \cup \{aa, aab, aba, abab\} \cup \dots$$

Note : ϵ is in L^* , for every language L , including $\emptyset$.

The previously introduced definition of Σ^* is an instance of Kleene star.

Finite Automata

DFA & NFA

Finite Automata

Automata (singular : automaton) are a particularly simple, but useful, model of computation. They were initially proposed as a simple model for the behavior of neurons. The concept of a finite automaton appears to have arisen in the 1943 paper "A logical calculus of the ideas immanent in nervous activity", by Warren McCulloch and Walter Pitts. In 1951 Kleene introduced regular expressions to describe the behaviour of finite automata. He also proved the important theorem saying that regular expressions exactly capture the behaviours of finite automata. In 1959, Dana Scott and Michael Rabin introduced non-deterministic automata and showed the surprising theorem that they are equivalent to deterministic automata. We will study these fundamental results. Since those early years, the study of automata has continued to grow, showing that they are indeed a fundamental idea in computing.

States, Transitions and Finite-State Transition System :

Let us first give some intuitive idea about a state of a system and state transitions before describing finite automata.

Informally, a state of a system is an instantaneous description of that system which gives all relevant information necessary to determine how the system can evolve from that point on.

Transitions are changes of states that can occur spontaneously or in response to inputs to the states. Though transitions usually take time, we assume that state transitions are instantaneous (which is an abstraction).

Some examples of state transition systems are: digital systems, vending machines, etc.

A system containing only a finite number of states and transitions among them is called a finite-state transition system.

Finite-state transition systems can be modeled abstractly by a mathematical model called finite automation.

We said that automata are a model of computation. That means that they are a simplified abstraction of 'the real thing'. So what gets abstracted away? One thing that disappears is any notion of hardware or software. We merely deal with states and transitions between states. The distinction between program and machine executing it disappears. One could say that an automaton is the machine and the program. This makes automata relatively easy to implement in either hardware or software. From the point of view of resource consumption, the essence of a finite automaton is that it is a strictly finite model of computation. Everything in it is of a fixed, finite size and cannot be modified in the course of the computation.

Finite Automaton can be classified into two types

- **Deterministic Finite Automaton (DFA)**
- **Non-deterministic Finite Automaton (NDFA / NFA)**

Deterministic Finite (-state) Automata

Informally, a DFA (Deterministic Finite State Automaton) is a simple machine that reads an input string -- one symbol at a time -- and then, after the input has been completely read, decides whether to accept or reject the input. As the symbols are read from the tape, the automaton can change its state, to reflect how it reacts to what it has seen so far.

Thus, a DFA conceptually consists of 3 parts:

1. A **tape** to hold the input string. The tape is divided into a finite number of cells. Each cell holds a symbol from Σ .
2. A **tape head** for reading symbols from the tape
3. A **control**, which itself consists of 3 things:
 - ♦ finite number of states that the machine is allowed to be in (zero or more states are designated as **accept** or **final** states),
 - ♦ a current state, initially set to a start state,
 - ♦ a state transition function for changing the current state.

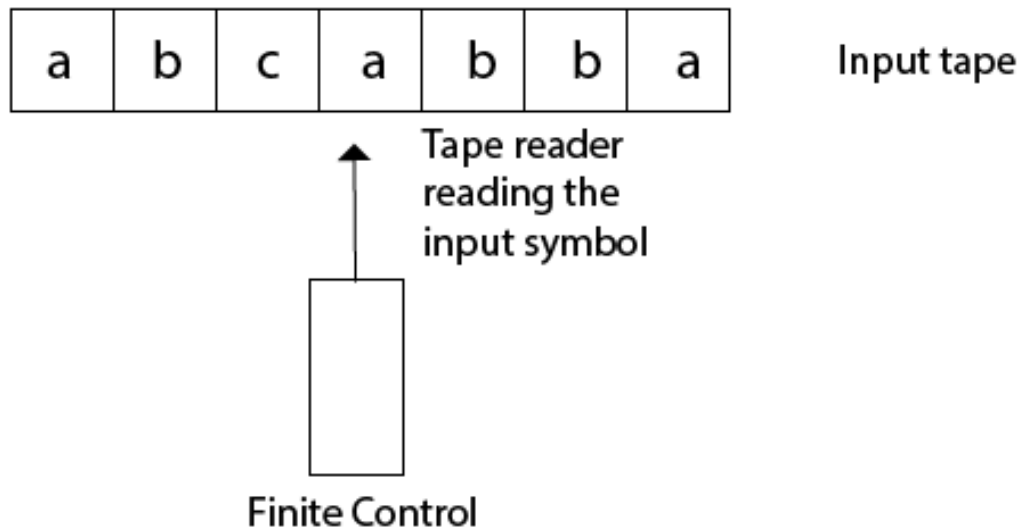


Fig :- Finite automata model

An automaton processes a string on the tape by repeating the following actions until the tape head has traversed the entire string:

1. The tape head reads the current tape cell and sends the symbol s found there to the control. Then the tape head moves to the next cell.
2. The control takes s and the current state and consults the state transition function to get the next state, which becomes the new current state.

Once the entire string has been processed, the state in which the automation enters is examined. If it is an **accept state**, the input string is **accepted**; otherwise, the string is **rejected**. Summarizing all the above we can formulate the following formal definition:

Deterministic Finite State Automaton : A **Deterministic Finite State Automaton** (DFA) is a 5-tuple :

$$M = (Q, \Sigma, \delta, q_0, F)$$

- Q is a finite set of states.
- Σ is a finite set of input symbols or alphabet.

- $\delta: Q \times \Sigma \rightarrow Q$ is the "next state" transition function (which is total). Intuitively, δ is a function that tells which state to move to in response to an input, i.e., if M is in state q and sees input a , it moves to state $\delta(q, a)$.
- $q_0 \in Q$ is the start state.
- $F \subseteq Q$ is the set of accept or final states.

Acceptance of Strings :

A DFA accepts a string $w = a_1 a_2 \dots a_n$ if there is a sequence of states $q_0, q_1, \dots, q_n$ in Q such that

1. q_0 is the start state.
2. $\delta(q_i, a_{i+1}) = q_{i+1}$ for all $0 \leq i < n$.
3. $q_n \in F$.

Language Accepted or Recognized by a DFA :

The language accepted or recognized by a DFA M is the set of all strings accepted by M , and is denoted by $L(M)$ i.e. $L(M) = \{w \in \Sigma^* \mid M \text{ accepts } w\}$.

The notion of acceptance can also be made more precise by extending the transition function δ .

Extended transition function :

Extend $\delta : Q \times \Sigma \rightarrow Q$ (which is function on symbols) to a function on strings, i.e. $\delta^* : Q \times \Sigma^* \rightarrow Q$.

That is, $\delta^*(q, w)$ is the state the automation reaches when it starts from the state q and finish processing the string w . Formally, we can give an inductive definition as follows:

The language of the DFA M is the set of strings that can take the start state to one of the accepting states i.e.

$$\begin{aligned} L(M) &= \{ w \in \Sigma^* \mid M \text{ accepts } w \} \\ &= \{ w \in \Sigma^* \mid \delta^*(q_0, w) \in F \} \end{aligned}$$

Example 1 :

$$M = (Q, \Sigma, \delta, q_0, F)$$

$$Q = \{q_0, q_1\}$$

q_0 is the start state

$$F = \{q_1\}$$

$$\delta(q_0, 0) = q_0 \quad \delta(q_1, 0) = q_1$$

$$\delta(q_0, 1) = q_1 \quad \delta(q_1, 1) = q_1$$

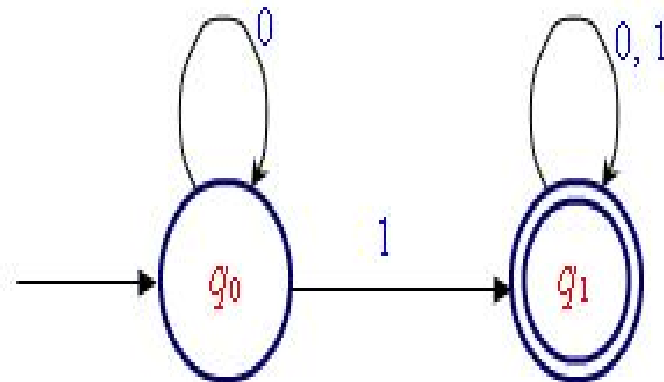
It is a formal description of a DFA. But it is hard to comprehend. For ex. The language of the DFA is any string over $\{0, 1\}$ having at least one 1.

We can describe the same DFA by transition table or state transition diagram as following:

Transition Table :

	0	1
$\rightarrow q_0$	q_0	q_1
$* q_1$	q_1	q_1

State transition diagram :



It is easy to comprehend the transition diagram.

Explanation : We cannot reach final state q_1 w/o 1 in the i/p string. There can be any no. of 0's at the beginning. (The self-loop at q_0 on label 0 indicates it). Similarly there can be any no. of 0's & 1's in any order at the end of the string.

Transition table :

It is basically a tabular representation of the transition function that takes two arguments (a state and a symbol) and returns a value (the "next state").

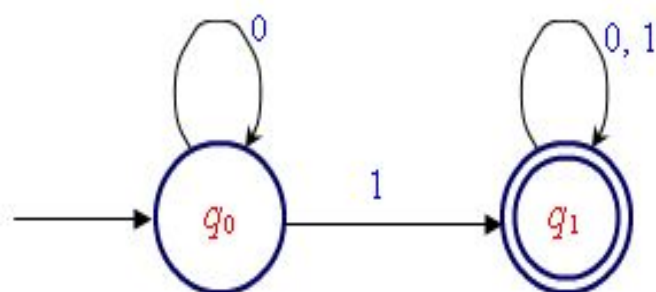
- Rows correspond to states,
- Columns correspond to input symbols,
- Entries correspond to next states
- The start state is marked with an arrow
- The accept states are marked with a star (*).

	0	1
$\rightarrow q_0$	q_0	q_1
$* q_1$	q_1	q_1

(State) Transition diagram :

A state transition diagram or simply a transition diagram is a directed graph which can be constructed as follows:

1. For each state in Q there is a node.
2. There is a directed edge from node q to node p labeled a iff $\delta(q, a) = p$. (If there are several input symbols that cause a transition, the edge is labeled by the list of these symbols.)
3. There is an arrow with no source into the start state.
4. Accepting states are indicated by double circle.



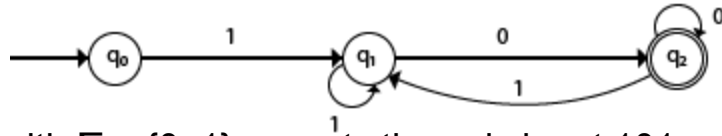
Here is an informal description how a DFA operates. An input to a DFA can be any string $w \in \Sigma^*$. Put a pointer to the start state q . Read the input string w from left to right, one symbol at a time, moving the pointer according to the transition function, δ . If the next symbol of w is a and the pointer is on state p , move the pointer to $\delta(p, a)$. When the end of the input string w is encountered, the pointer is on some state, r . The string is said to be accepted by the DFA if $r \in F$ and rejected if $r \notin F$. Note that there is no formal mechanism for moving the pointer.

A language $L \in \Sigma^*$ is said to be regular if $L = L(M)$ for some DFA M .

Examples of DFA

1. Design a FA with $\Sigma = \{0, 1\}$ accepts those string which starts with 1 and ends with 0.
2. Design a FA with $\Sigma = \{0, 1\}$ accepts the only input 101.
3. Design FA with $\Sigma = \{0, 1\}$ accepts even number of 0's and even number of 1's.
4. Design FA with $\Sigma = \{0, 1\}$ accepts the set of all strings with three consecutive 0's.
5. Design a FA with $\Sigma = \{0, 1\}$ accepts those string which starts with 0 and ends with 1
6. Design FA with $\Sigma = \{0, 1\}$ accepts the set of all strings always ends with 00.
7. Design a DFA $L(M) = \{w \mid w \in \{0, 1\}^*\}$ and W is a string that does not contain consecutive 1's.
8. Design a FA with $\Sigma = \{0, 1\}$ accepts the strings with an even number of 0's followed by single 1.
9. Design a DFA which checks whether the given binary number is even.
10. Design a FA with $\Sigma = \{0, 1\}$ accepts the strings with odd number of 1's and any number of 0's.
11. Design a DFA which checks whether the given binary number is divisible by 3.
12. Design a DFA which checks whether the given decimal number is divisible by 3.
13. Design a DFA to accept L , where $L = \{\text{Strings in which a always appears trippled}\}$ over the set $\Sigma = \{a, b\}$.
14. Design a DFA to accept L , where $L = \{\text{Strings in } L \text{ such that total number of a's in thm are divisble by 3}\}$ over the set $\Sigma = \{a, b\}$.

Design a FA with $\Sigma = \{0, 1\}$ accepts those string which starts with 1 and ends with 0.



Design a FA with $\Sigma = \{0, 1\}$ accepts the only input 101.

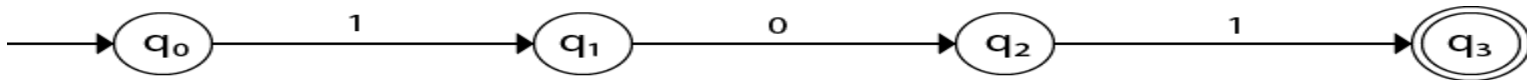
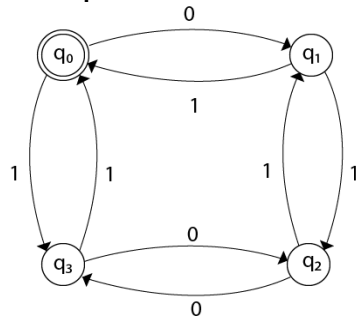
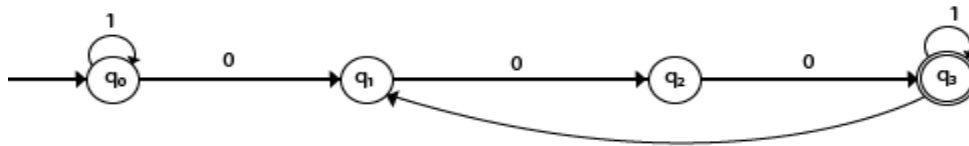


Fig: FA

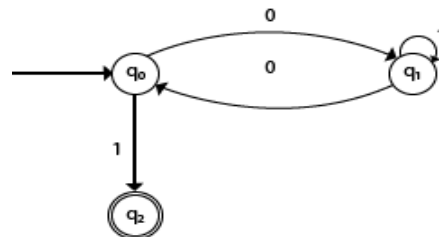
Design FA with $\Sigma = \{0, 1\}$ accepts even number of 0's and even number of 1's.



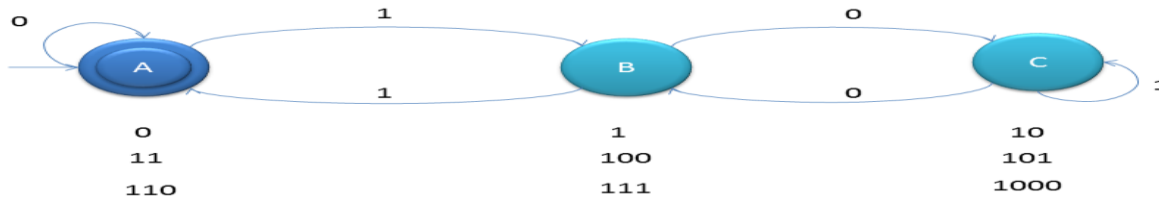
Design FA with $\Sigma = \{0, 1\}$ accepts the set of all strings with three consecutive 0's.



Design a FA with $\Sigma = \{0, 1\}$ accepts the strings with an even number of 0's followed by single 1.

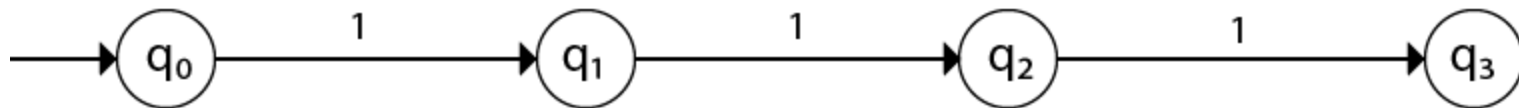


- Design a DFA which checks whether the given binary number is divisible by 3.

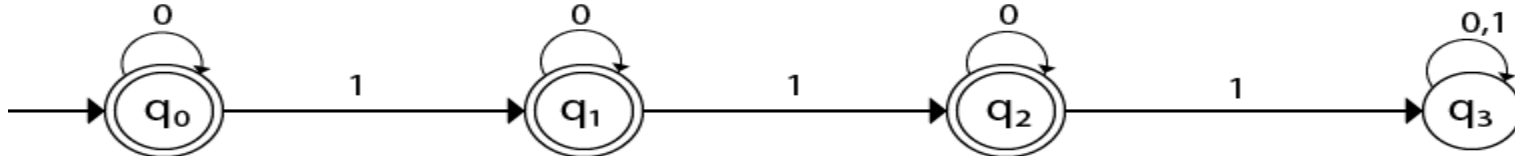


Design a DFA $L(M) = \{w \mid w \in \{0, 1\}^*\}$ and W is a string that does not contain consecutive 1's.

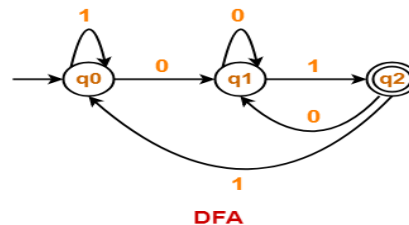
When three consecutive 1's occur the DFA will be:



Here two consecutive 1's or single 1 is acceptable, hence



Design FA with $\Sigma = \{0, 1\}$ accepts the set of all strings end with 01



Non Deterministic Finite Automata

In NDFA, for a particular input symbol, the machine can move to any combination of the states in the machine. In other words, the exact state to which the machine moves cannot be determined. Hence, it is called **Non-deterministic Automaton**. As it has finite number of states, the machine is called **Non-deterministic Finite Machine** or **Non-deterministic Finite Automaton**.

Formal Definition of an NDFA

An NDFA can be represented by a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where –
Q is a finite set of states.

Σ is a finite set of symbols called the alphabets.

δ is the transition function where $\delta: Q \times \Sigma \rightarrow 2^Q$

(Here the power set of Q (2^Q) has been taken because in case of NDFA, from a state, transition can occur to any combination of Q states)

q_0 is the initial state from where any input is processed ($q_0 \in Q$).

F is a set of final state/states of Q ($F \subseteq Q$).

Multiple Next State :

In contrast to a DFA, the next state is not necessarily uniquely determined by the current state and input symbol in case of an NFA. (Recall that, in a DFA there is exactly one start state and exactly one transition out of every state for each symbol in Σ).

This means that - in a state q and with input symbol a - there could be one, more than one or zero next state to go, i.e. the value of $\delta(q, a)$ is a subset of Q . Thus $\delta(q, a) = \{q_1, q_2, \dots, q_k\}$ which means that any one of $q_1, q_2, \dots, q_k$ could be the next state.

The zero next state case is a special one giving $\delta(q, a) = \phi$, which means that there is no next state on input symbol when the automata is in state q . In such a case, we may think that the automata "hangs" and the input will be rejected.

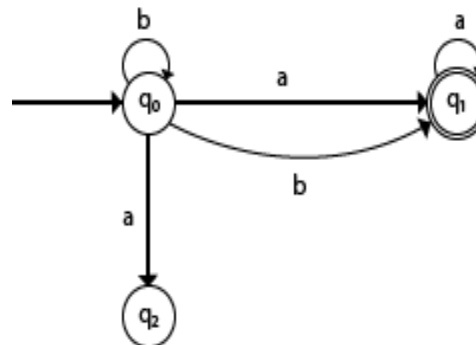


Fig:- NDFA

ε - transitions :

In an ε-transition, the tape head doesn't do anything- it doesnot read and it doesnot move. However, the state of the automata can be changed - that is can go to zero, one or more states. This is written formally as $\delta(q, \epsilon) = \{q_1, q_2, \dots, q_k\}$ implying that the next state could be any one of $q_1, q_2, \dots, q_k$ w/o consuming the next input symbol.

Acceptance :

Informally, an NFA is said to accept its input w if it is possible to start in some start state and process w , moving according to the transition rules and making choices along the way whenever the next state is not uniquely defined, such that when w is completely processed (i.e. end of w is reached), the automata is in an accept state. There may be several possible paths through the automation in response to an input w since the start state is not determined and there are choices along the way because of multiple next states. Some of these paths may lead to accept states while others may not. The automation is said to accept w if at least one computation path on input w starting from at least one start state leads to an accept state- otherwise, the automation rejects input w . Alternatively, we can say that, w is accepted iff there exists a path with label w from some start state to some accept state. Since there is no mechanism for determining which state to start in or which of the possible next moves to take (including the ε-transitions) in response to an input symbol we can think that the automation is having some "guessing" power to choose the correct one in case the input is accepted.

Example 1 : Consider the language $L = \{w \in \{0, 1\}^* \mid \text{The 3rd symbol from the right is 1}\}$. The following four-state automaton accepts L .

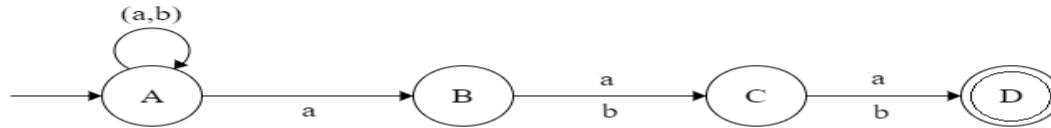
The m/c is not deterministic since there are two transitions from state q_1 on input 1 and no transition (zero transition) from q_4 on both 0 & 1.

For any string w whose 3rd symbol from the right is a 1, there exists a sequence of legal transitions leading from the start state q_1 to the accept state q_4 . But for any string w where 3rd symbol from the right is 0, there is no possible sequence of legal transitions leading from q_1 and q_4 . Hence m/c accepts L . How does it accept any string $w \in L$?

The m/c starts at q_1 and remains in the state q_1 on any input until the 3rd symbol from the right is encountered. (Of course, w must satisfy $|w| \geq 3$). At this point, if the symbol is 1, it goes to the state q_2 and then enters q_3 & q_4 in the next two steps on any input 0 or 1. But if the 3rd symbol from the right is 0, thus it will get stuck at that point, because of no transition defined.

To enter the state q_2 from q_1 , the m/c needs the input 1. If the 1 occur prior to the position 4 in the input or more from the right (instead of 3rd), thus it can enter q_2 from q_1 on that input and finally will enter accept state q_4 but at that point some of the input symbols may be left i.e. the input will not be exhausted and hence, the string will not be accepted by the m/c.

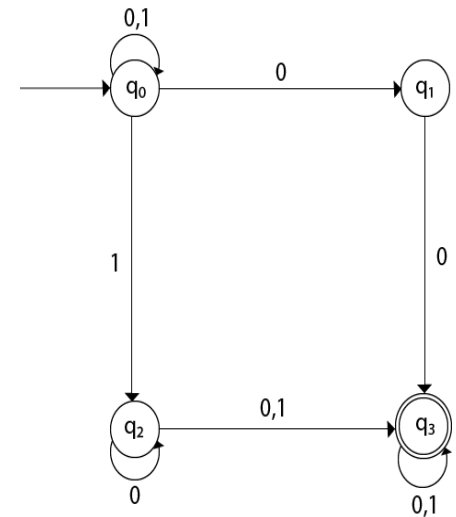
1. The NFA of the language containing all the strings in which 3rd symbol from the RHS is “a”.



Required NFA

2. Design a NFA for the transition table as given below:

Present State	0	1
$\rightarrow q_0$	q_0, q_1	q_0, q_2
q_1	q_3	ϵ
q_2	q_2, q_3	q_3
$\rightarrow q_3$	q_3	q_3



Here, the transition functions are shown:

$\delta(q_0, 0) = \{q_0, q_1\}$, $\delta(q_0, 1) = \{q_0, q_2\}$

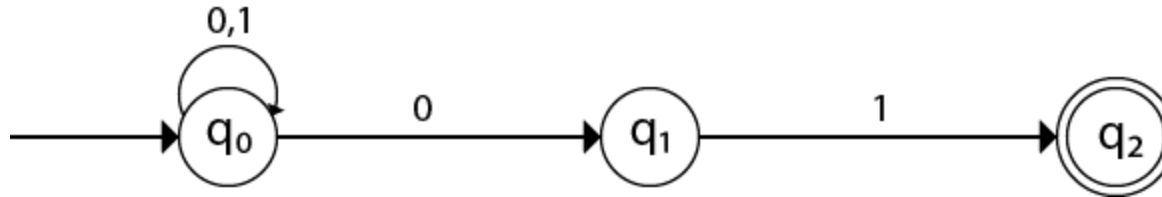
Then, $\delta(q_1, 0) = \{q_3\}$, Then, $\delta(q_2, 0) = \{q_2, q_3\}$

$\delta(q_2, 1) = \{q_3\}$, Then, $\delta(q_3, 0) = \{q_3\}$

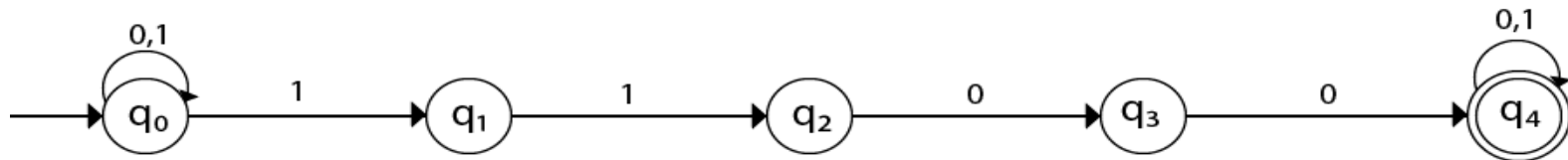
$\delta(q_3, 1) = \{q_3\}$

Examples of NFA

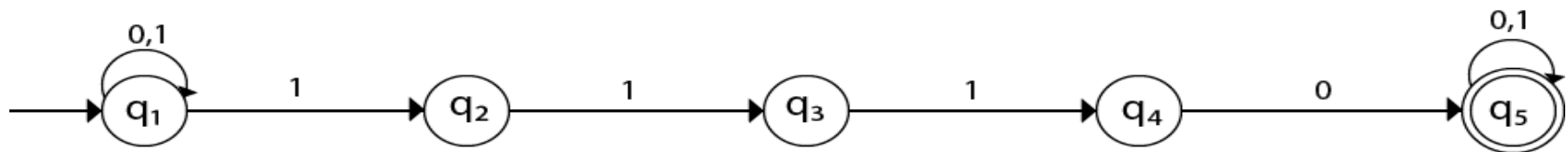
Design an NFA with $\Sigma = \{0, 1\}$ accepts all string ending with 01.



Design an NFA with $\Sigma = \{0, 1\}$ in which double '1' is followed by double '0'.



Design an NFA in which all the string contain a substring 1110.



Consider a string 111010,
 $\delta(q1, 111010) = \delta(q1, 1100)$
 $= \delta(q1, 100)$
 $= \delta(q2, 00)$

$\delta(q1, 111010) = \delta(q2, 1100)$
 $= \delta(q3, 100)$
 $= \delta(q4, 00)$
 $= \delta(q5, 0)$
 $= \delta(q5, \epsilon)$

NFA with Epsilon(ϵ) Transitions

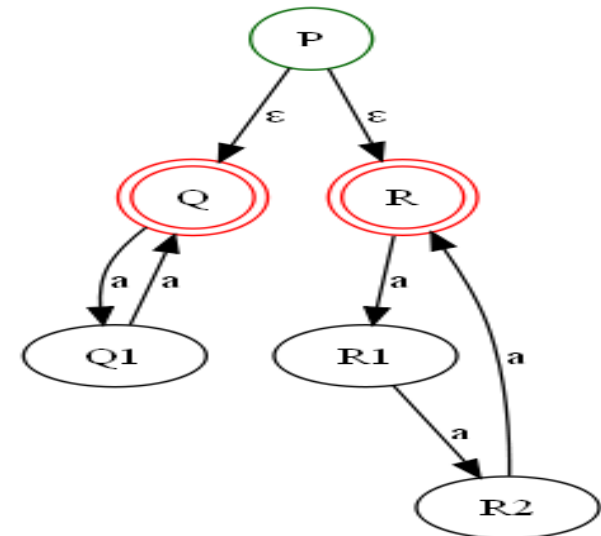
A ϵ -NFA is a quintuple $A=(Q,\Sigma,\delta,q_0,F)$

where

- Q is a set of states
- Σ is the alphabet of input symbols
- $q_0 \in Q$ is the initial state
- $F \subseteq Q$ is the set of final states
- $\delta: Q \times \Sigma_\epsilon \rightarrow 2^Q$ is the transition function
 - Note ϵ is never a member of Σ
 - Σ_ϵ is defined to be $(\Sigma \cup \epsilon)$

Example of NFA with Epsilon Transitions

- $\{ a^n \mid n \text{ is even or divisible by } 3 \}$



ϵ -Closure

- ϵ -closure of a state
- The ϵ -closure of the state q , denoted $\epsilon\text{-Closure}(q)$, is the set that contains q , together with all states that can be reached starting at q by following only ϵ -transitions.

Computing ϵ -Closure

Compute ϵ -Closure by adding new states until no new states can be added

Example:

Start with $\{P\}$

Add Q and R to get $\{P, Q, R\}$

Add S to get $\{P, Q, R, S\}$

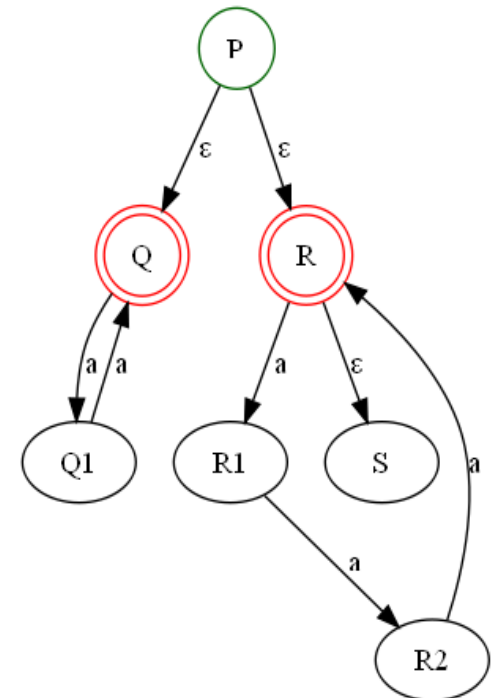
No new states can be added

$\epsilon\text{-Closure}(P) = \{P, Q, R, S\}$

$\epsilon\text{-Closure}(R) = \{R, S\}$

$\epsilon\text{-Closure}(Q) = \{Q\}$

$\epsilon\text{-Closure}(S) = \{S\}$



Removing ϵ -Transitions

NFA with ϵ can be converted to NFA without ϵ , and this NFA without ϵ can be converted to DFA. To do this, we will use a method, which can remove all the ϵ transition from given NFA. The method will be:

- Find out all the ϵ transitions from each state from Q . That will be called as ϵ -closure $\{q_1\}$ where $q_i \in Q$.
- Then δ' transitions can be obtained. The δ' transitions mean a ϵ -closure on δ moves.

Rule for conversion:

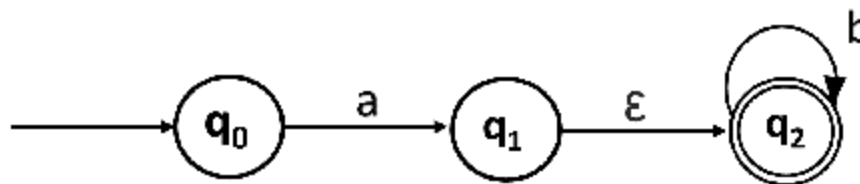
$\delta'(q, a) = \epsilon\text{-closure}(\delta(\delta^*(q, \epsilon), a))$, where $\delta^*(q, \epsilon) = \epsilon\text{-closure}(q)$

- Repeat Step-2 for each input symbol and each state of given NFA.
- Using the resultant states, the transition table for equivalent NFA without ϵ can be built.

Example:

Convert the following NFA with ϵ to NFA without ϵ .

Solutions: We will first obtain ϵ -closures of q_0 , q_1 and q_2 as follows:



$$\varepsilon\text{-closure}(q_0) = \{q_0\}$$

$$\varepsilon\text{-closure}(q_1) = \{q_1, q_2\}$$

$$\varepsilon\text{-closure}(q_2) = \{q_2\}$$

Step 2: Now the δ' transition on each input symbol is obtained as:

$$\begin{aligned}\delta'(q_0, a) &= \varepsilon\text{-closure}(\delta(\delta^*(q_0, \varepsilon), a)) \\ &= \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(q_0), a)) \\ &= \varepsilon\text{-closure}(\delta(q_0, a)) \\ &= \varepsilon\text{-closure}(q_1) \\ &= \{q_1, q_2\}\end{aligned}$$

$$\begin{aligned}\delta'(q_0, b) &= \varepsilon\text{-closure}(\delta(\delta^*(q_0, \varepsilon), b)) \\ &= \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(q_0), b)) \\ &= \varepsilon\text{-closure}(\delta(q_0, b)) \\ &= \Phi\end{aligned}$$

- Now the δ' transition on q_1 is obtained as:

$$\begin{aligned}\delta'(q_1, a) &= \varepsilon\text{-closure}(\delta(\delta^*(q_1, \varepsilon), a)) \\ &= \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(q_1), a)) \\ &= \varepsilon\text{-closure}(\delta(q_1, q_2), a) \\ &= \varepsilon\text{-closure}(\delta(q_1, a) \cup \delta(q_2, a)) \\ &= \varepsilon\text{-closure}(\Phi \cup \Phi) \\ &= \Phi\end{aligned}$$

$$\begin{aligned}
\delta'(q1, b) &= \varepsilon\text{-closure}(\delta(\delta^{\wedge}(q1, \varepsilon), b)) \\
&= \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(q1), b)) \\
&= \varepsilon\text{-closure}(\delta(q1, q2), b) \\
&= \varepsilon\text{-closure}(\delta(q1, b) \cup \delta(q2, b)) \\
&= \varepsilon\text{-closure}(\Phi \cup q2) \\
&= \{q2\}
\end{aligned}$$

- The δ' transition on $q2$ is obtained as:

$$\begin{aligned}
\delta'(q2, a) &= \varepsilon\text{-closure}(\delta(\delta^{\wedge}(q2, \varepsilon), a)) \\
&= \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(q2), a)) \\
&= \varepsilon\text{-closure}(\delta(q2, a)) \\
&= \varepsilon\text{-closure}(\Phi) \\
&= \Phi
\end{aligned}$$

$$\begin{aligned}
\delta'(q2, b) &= \varepsilon\text{-closure}(\delta(\delta^{\wedge}(q2, \varepsilon), b)) \\
&= \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(q2), b)) \\
&= \varepsilon\text{-closure}(\delta(q2, b)) \\
&= \varepsilon\text{-closure}(q2) \\
&= \{q2\}
\end{aligned}$$

Now we will summarize all the computed δ' transitions:

$$\delta'(q_0, a) = \{q_0, q_1\}$$

$$\delta'(q_0, b) = \Phi$$

$$\delta'(q_1, a) = \Phi$$

$$\delta'(q_1, b) = \{q_2\}$$

$$\delta'(q_2, a) = \Phi$$

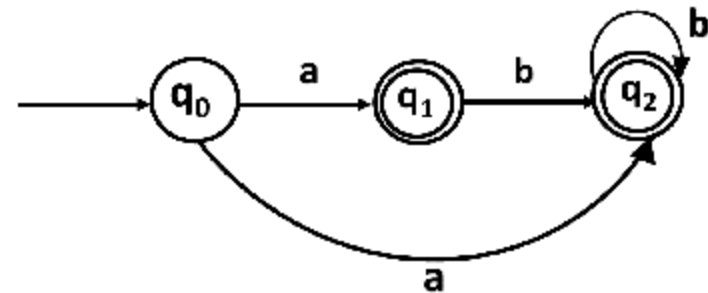
$$\delta'(q_2, b) = \{q_2\}$$

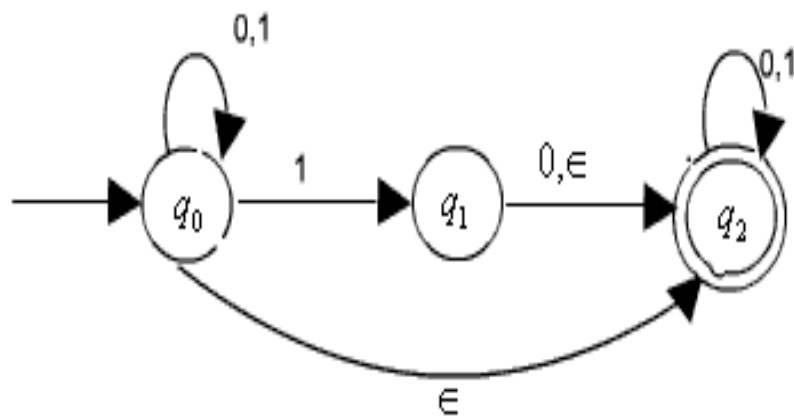
The transition table can be:

States	a	b
$\rightarrow q_0$	$\{q_1, q_2\}$	Φ
$*q_1$	Φ	$\{q_2\}$
$*q_2$	Φ	$\{q_2\}$

States	a	b
$\rightarrow q_0$	$[q_1, q_2]$	Φ
$*q_1$	Φ	$[q_2]$
$*q_2$	Φ	$[q_2]$
$*[q_1, q_2]$	Φ	$[q_2]$

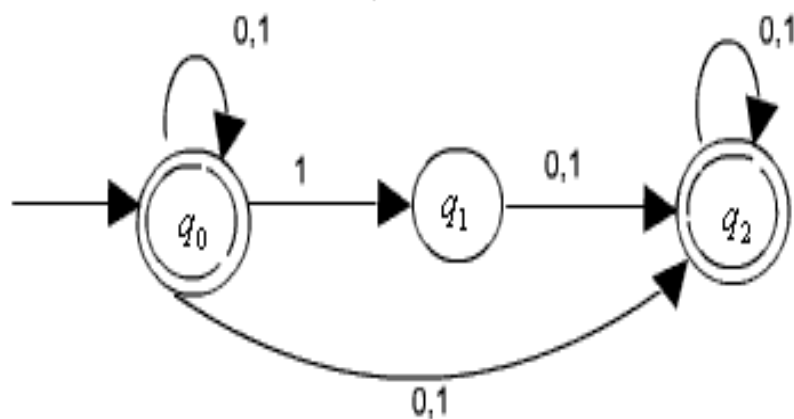
State q_1 and q_2 become the final state as ϵ -closure of q_1 and q_2 contain the final state q_2 . The NFA can be shown by the following transition diagram:





	0	1	ε
$\rightarrow q_0$	$\{q_0\}$	$\{q_0, q_1\}$	$\{q_2\}$
q_1	$\{q_2\}$	$\emptyset$	$\{q_2\}$
	$\{q_2\}$	$\emptyset$	$\{q_2\}$

Transition table δ



	0	1
$\xrightarrow{F} q_0$	$\{q_0, q_2\}$	$\{q_0, q_1, q_2\}$
q_1	$\{q_2\}$	$\{q_2\}$
	$\{q_2\}$	$\{q_2\}$

Transition table δ' for the equivalent NFA without ϵ - moves

Conversion from NFA to DFA

Let, $M = (Q, \Sigma, \delta, q_0, F)$ is an NFA which accepts the language $L(M)$. There should be equivalent DFA denoted by $M' = (Q', \Sigma', q_0', \delta', F')$ such that $L(M) = L(M')$.

Steps for converting NFA to DFA:

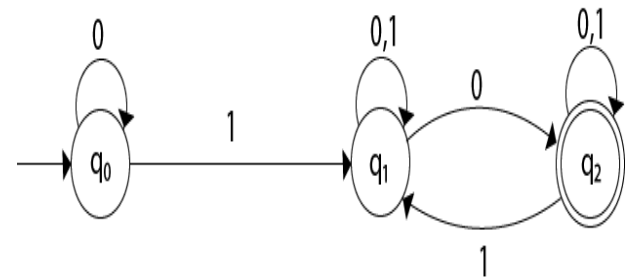
Step 1: Initially $Q' = \phi$

Step 2: Add q_0 of NFA to Q' . Then find the transitions from this start state.

Step 3: In Q' , find the possible set of states for each input symbol. If this set of states is not in Q' , then add it to Q' .

Step 4: In DFA, the final state will be all the states which contain F (final states of NFA)

State	0	1
$\rightarrow q_0$	q_0	q_1
q_1	$\{q_1, q_2\}$	q_1
$*q_2$	q_2	$\{q_1, q_2\}$



Now we will obtain δ' transition for state q_0 .

$\delta'([q_0], 0) = [q_0]$

$\delta'([q_0], 1) = [q_1]$

The δ' transition for state q_1 is obtained as:

$\delta'([q_1], 0) = [q_1, q_2]$ (**new** state generated)

$\delta'([q_1], 1) = [q_1]$

The δ' transition for state q_2 is obtained as:

$\delta'([q_2], 0) = [q_2]$

$\delta'([q_2], 1) = [q_1, q_2]$

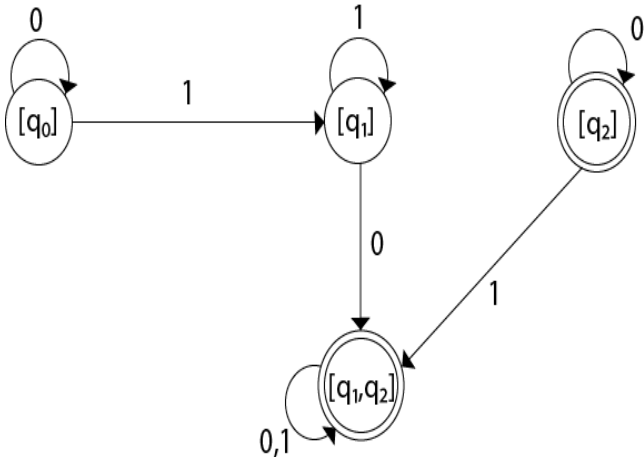
Now we will obtain δ' transition on $[q_1, q_2]$.

$$\begin{aligned} \delta'([q_1, q_2], 0) &= \delta(q_1, 0) \cup \delta(q_2, 0) \\ &= \{q_1, q_2\} \cup \{q_2\} \\ &= [q_1, q_2] \end{aligned}$$

$$\begin{aligned} \delta'([q_1, q_2], 1) &= \delta(q_1, 1) \cup \delta(q_2, 1) \\ &= \{q_1\} \cup \{q_1, q_2\} \\ &= \{q_1, q_2\} \\ &= [q_1, q_2] \end{aligned}$$

The state $[q_1, q_2]$ is the final state as well because it contains a final state q_2 . The transition table for the constructed DFA will be:

State	0	1
$\rightarrow [q_0]$	$[q_0]$	$[q_1]$
$[q_1]$	$[q_1, q_2]$	$[q_1]$
$*[q_2]$	$[q_2]$	$[q_1, q_2]$
$*[q_1, q_2]$	$[q_1, q_2]$	$[q_1, q_2]$



Conversion from NFA with ϵ to DFA

Steps for converting NFA with ϵ to DFA:

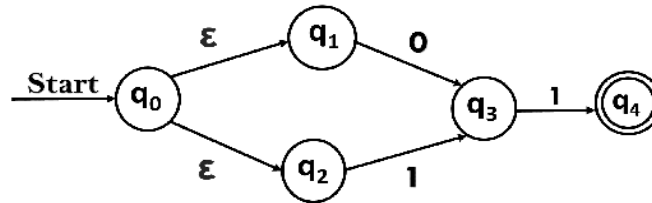
Step 1: We will take the ϵ -closure for the starting state of NFA as a starting state of DFA.

Step 2: Find the states for each input symbol that can be traversed from the present. That means the union of transition value and their closures for each state of NFA present in the current state of DFA.

Step 3: If we found a new state, take it as current state and repeat step 2.

Step 4: Repeat Step 2 and Step 3 until there is no new state present in the transition table of DFA.

Step 5: Mark the states of DFA as a final state which contains the final state of NFA.



Let us obtain ϵ -closure of each state.

1. ϵ -closure $\{q_0\} = \{q_0, q_1, q_2\}$

2. ϵ -closure $\{q_1\} = \{q_1\}$

3. ϵ -closure $\{q_2\} = \{q_2\}$

4. ϵ -closure $\{q_3\} = \{q_3\}$

5. ϵ -closure $\{q_4\} = \{q_4\}$

Now, let ε -closure $\{q_0\} = \{q_0, q_1, q_2\}$ be state A.

Hence

$$\begin{aligned}\delta'(A, 0) &= \varepsilon\text{-closure } \{\delta((q_0, q_1, q_2), 0)\} \\ &= \varepsilon\text{-closure } \{\delta(q_0, 0) \cup \delta(q_1, 0) \cup \delta(q_2, 0)\} \\ &= \varepsilon\text{-closure } \{q_3\} \\ &= \{q_3\} \quad \text{call it as state B.}\end{aligned}$$

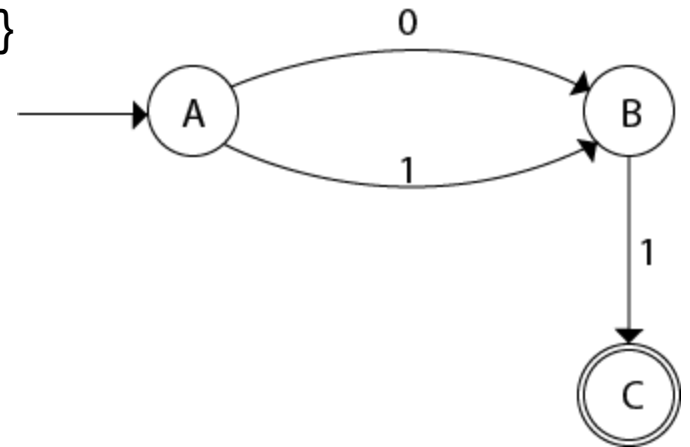
$$\begin{aligned}\delta'(A, 1) &= \varepsilon\text{-closure } \{\delta((q_0, q_1, q_2), 1)\} \\ &= \varepsilon\text{-closure } \{\delta(q_0, 1) \cup \delta(q_1, 1) \cup \delta(q_2, 1)\} \\ &= \varepsilon\text{-closure } \{q_3\} \\ &= \{q_3\} = B.\end{aligned}$$

Now,

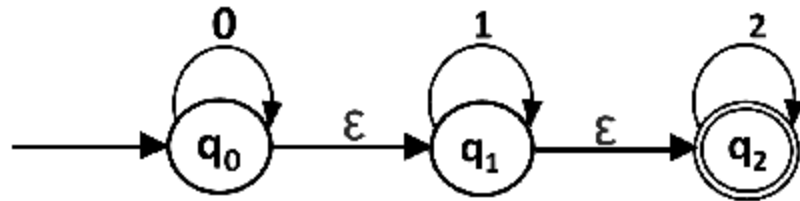
$$\begin{aligned}\delta'(B, 0) &= \varepsilon\text{-closure } \{\delta(q_3, 0)\} \\ &= \phi \\ \delta'(B, 1) &= \varepsilon\text{-closure } \{\delta(q_3, 1)\} \\ &= \varepsilon\text{-closure } \{q_4\} \\ &= \{q_4\} \quad \text{i.e. state C}\end{aligned}$$

For state C:

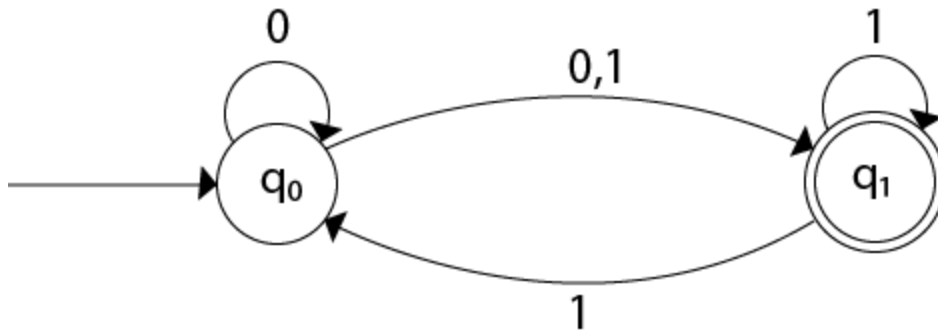
$$\begin{aligned}\delta'(C, 0) &= \varepsilon\text{-closure } \{\delta(q_4, 0)\} \\ &= \phi \\ \delta'(C, 1) &= \varepsilon\text{-closure } \{\delta(q_4, 1)\} \\ &= \phi\end{aligned}$$



Convert the given NFA into its equivalent DFA.



Convert the given NFA to DFA.



Minimization of DFA

Minimization of DFA means reducing the number of states from given FA. Thus, we get the FSM(finite state machine) with redundant states after minimizing the FSM.

We have to follow the various steps to minimize the DFA. These are as follows:

Step 1: Remove all the states that are unreachable from the initial state via any set of the transition of DFA.

Step 2: Draw the transition table for all pair of states.

Step 3: Now split the transition table into two tables T1 and T2. T1 contains all final states, and T2 contains non-final states.

Step 4: Find similar rows from T1 such that:

1. $\delta(q, a) = p$

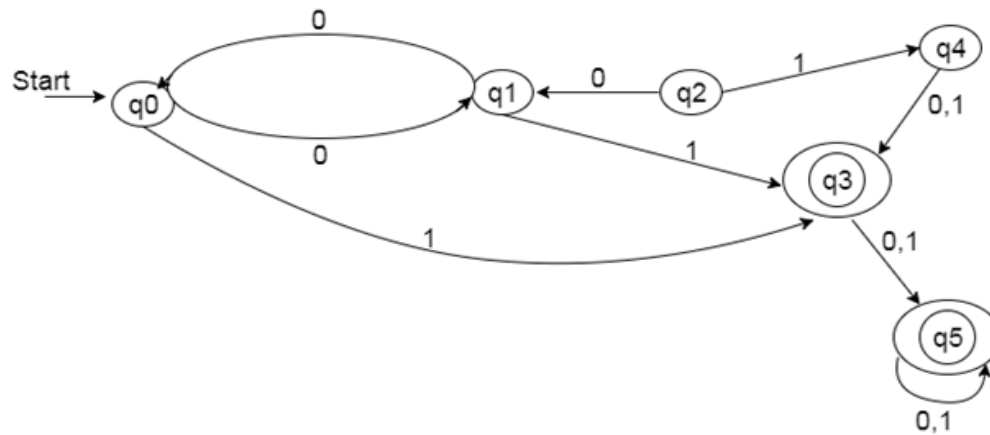
2. $\delta(r, a) = p$

That means, find the two states which have the same value of a and b and remove one of them.

Step 5: Repeat step 3 until we find no similar rows available in the transition table T1.

Step 6: Repeat step 3 and step 4 for table T2 also.

Step 7: Now combine the reduced T1 and T2 tables. The combined transition table is the transition table of minimized DFA.



Solution:

Step 1: In the given DFA, q2 and q4 are the unreachable states so remove them.

Step 2: Draw the transition table for the rest of the states.

State	0	1
→q0	q1	q3
q1	q0	q3
*q3	q5	q5
*q5	q5	q5

Step 3: Now divide rows of transition table into two sets as:

1. One set contains those rows, which start from non-final states:

State	0	1
q0	q1	q3
q1	q0	q3

2. Another set contains those rows, which starts from final states.

State	0	1
q3	q5	q5
q5	q5	q5

Step 4: Set 1 has no similar rows so set 1 will be the same.

Step 5: In set 2, row 1 and row 2 are similar since q3 and q5 transit to the same state on 0 and 1. So skip q5 and then replace q5 by q3 in the rest.

State	0	1
q3	q3	q3

Step 6: Now combine set 1 and set 2 as:

State	0	1
$\rightarrow q0$	q1	q3
q1	q0	q3
*q3	q3	q3

